

work, span, and parallelism of the resulting algorithm.

26.1-10

For what number of processors do the two versions of the chess program run equally fast, assuming that $T_P = T_1/P + T_\infty$?

26.2 Parallel matrix multiplication

In this section, we'll explore how to parallelize the three matrix-multiplication algorithms from Sections 4.1 and 4.2. We'll see that each algorithm can be parallelized in a straightforward fashion using either parallel loops or recursive spawning. We'll analyze them using work/span analysis, and we'll see that each parallel algorithm attains the same performance on one processor as its corresponding serial algorithm, while scaling up to large numbers of processors.

A parallel algorithm for matrix multiplication using parallel loops

The first algorithm we'll study is P-MATRIX-MULTIPLY, which simply parallelizes the two outer loops in the procedure MATRIX-MULTIPLY on page 81.

P-MATRIX-MULTIPLY (A, B, C, n)

```
1  parallel for  $i = 1$  to  $n$            // compute entries in each of  $n$  rows
2      parallel for  $j = 1$  to  $n$        // compute  $n$  entries in row  $i$ 
3          for  $k = 1$  to  $n$ 
4               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$  // add in another term of equation
                                           (4.1)
```

Let's analyze P-MATRIX-MULTIPLY. Since the serial projection of the algorithm is just MATRIX-MULTIPLY, the work is the same as the running time of MATRIX-MULTIPLY: $T_1(n) = \Theta(n^3)$. The span is $T_\infty(n) = \Theta(n)$, because it follows a path down the tree of recursion for the **parallel for** loop starting in line 1, then down the tree of recursion

for the **parallel for** loop starting in line 2, and then executes all n iterations of the ordinary **for** loop starting in line 3, resulting in a total span of $\Theta(\lg n) + \Theta(\lg n) + \Theta(n) = \Theta(n)$. Thus the parallelism is $\Theta(n^3)/\Theta(n) = \Theta(n^2)$. (Exercise 26.2-3 asks you to parallelize the inner loop to obtain a parallelism of $\Theta(n^3/\lg n)$, which you cannot do straightforwardly using **parallel for**, because you would create races.)

A parallel divide-and-conquer algorithm for matrix multiplication

Section 4.1 shows how to multiply $n \times n$ matrices serially in $\Theta(n^3)$ time using a divide-and-conquer strategy. Let's see how to parallelize that algorithm using recursive spawning instead of calls.

The serial MATRIX-MULTIPLY-RECURSIVE procedure on page 83 takes as input three $n \times n$ matrices A , B , and C and performs the matrix calculation $C = C + A \cdot B$ by recursively performing eight multiplications of $n/2 \times n/2$ submatrices of A and B . The P-MATRIX-MULTIPLY-RECURSIVE procedure on the following page implements the same divide-and-conquer strategy, but it uses spawning to perform the eight multiplications in parallel. To avoid determinacy races in updating the elements of C , it creates a temporary matrix D to store four of the submatrix products. At the end, it adds C and D together to produce the final result. (Problem 26-2 asks you to eliminate the temporary matrix D at the expense of some parallelism.)

Lines 2–3 of P-MATRIX-MULTIPLY-RECURSIVE handle the base case of multiplying 1×1 matrices. The remainder of the procedure deals with the recursive case. Line 4 allocates a temporary matrix D , and lines 5–7 zero it. Line 8 partitions each of the four matrices A , B , C , and D into $n/2 \times n/2$ submatrices. (As with MATRIX-MULTIPLY-RECURSIVE on page 83, we're glossing over the subtle issue of how to use index calculations to represent submatrix sections of a matrix.) The spawned recursive call in line 9 sets $C_{11} = C_{11} + A_{11} \cdot B_{11}$, so that C_{11} accumulates the first of the two terms in equation (4.5) on page 82. Similarly, lines 10–12 cause each of C_{12} , C_{21} , and C_{22} in parallel to accumulate the first of the two terms in equations (4.6)–(4.8),

respectively. Line 13 sets the submatrix D_{11} to the submatrix product $A_{12} \cdot B_{21}$, so that D_{11} equals the second of the two terms in equation (4.5). Lines 14–16 set each of D_{12} , D_{21} , and D_{22} in parallel to the second of the two terms in equations (4.6)–(4.8), respectively. The **sync** statement in line 17 ensures that all the spawned submatrix products in lines 9–16 have been computed, after which the doubly nested **parallel for** loops in lines 18–20 add the elements of D to the corresponding elements of C .

P-MATRIX-MULTIPLY-RECURSIVE (A, B, C, n)

```

1  if  $n == 1$                                 // just one element in each matrix?
2       $c_{11} = c_{11} + a_{11} \cdot b_{11}$ 
3      return
4  let  $D$  be a new  $n \times n$  matrix // temporary matrix
5  parallel for  $i = 1$  to  $n$                     // set  $D = 0$ 
6      parallel for  $j = 1$  to  $n$ 
7           $d_{ij} = 0$ 
8  partition  $A, B, C$ , and  $D$  into  $n/2 \times n/2$  submatrices  $A_{11}, A_{12}, A_{21},$ 
    $A_{22}; B_{11}, B_{12}, B_{21}, B_{22}; C_{11}, C_{12}, C_{21}, C_{22};$  and  $D_{11}, D_{12}, D_{21},$ 
    $D_{22};$  respectively
9  spawn P-MATRIX-MULTIPLY-RECURSIVE ( $A_{11}, B_{11}, C_{11},$ 
    $n/2$ )
10 spawn P-MATRIX-MULTIPLY-RECURSIVE ( $A_{11}, B_{12}, C_{12},$ 
    $n/2$ )
11 spawn P-MATRIX-MULTIPLY-RECURSIVE ( $A_{21}, B_{11}, C_{21},$ 
    $n/2$ )
12 spawn P-MATRIX-MULTIPLY-RECURSIVE ( $A_{21}, B_{12}, C_{22},$ 
    $n/2$ )
13 spawn P-MATRIX-MULTIPLY-RECURSIVE ( $A_{12}, B_{21}, D_{11},$ 
    $n/2$ )
14 spawn P-MATRIX-MULTIPLY-RECURSIVE ( $A_{12}, B_{22}, D_{12},$ 
    $n/2$ )

```

```

15 spawn P-MATRIX-MULTIPLY-RECURSIVE ( $A_{22}$ ,  $B_{21}$ ,  $D_{21}$ ,
     $n/2$ )
16 spawn P-MATRIX-MULTIPLY-RECURSIVE ( $A_{22}$ ,  $B_{22}$ ,  $D_{22}$ ,
     $n/2$ )
17 sync                                // wait for spawned submatrix products
18 parallel for  $i = 1$  to  $n$             // update  $C = C + D$ 
19     parallel for  $j = 1$  to  $n$ 
20          $c_{ij} = c_{ij} + d_{ij}$ 

```

Let's analyze the P-MATRIX-MULTIPLY-RECURSIVE procedure. We start by analyzing the work $M_1(n)$, echoing the serial running-time analysis of its progenitor MATRIX-MULTIPLY-RECURSIVE. The recursive case allocates and zeros the temporary matrix D in $\Theta(n^2)$ time, partitions in $\Theta(1)$ time, performs eight recursive multiplications of $n/2 \times n/2$ matrices, and finishes up with the $\Theta(n^2)$ work from adding two $n \times n$ matrices. Thus the work outside the spawned recursive calls is $\Theta(n^2)$, and the recurrence for the work $M_1(n)$ becomes

$$\begin{aligned}
 M_1(n) &= 8M_1(n/2) + \Theta(n^2) \\
 &= \Theta(n^3)
 \end{aligned}$$

by case 1 of the master theorem (Theorem 4.1). Not surprisingly, the work of this parallel algorithm is asymptotically the same as the running time of the procedure MATRIX-MULTIPLY on page 81, with its triply nested loops.

Let's determine the span $M_\infty(n)$ of P-MATRIX-MULTIPLY-RECURSIVE. Because the eight parallel recursive spawns all execute on matrices of the same size, the maximum span for any recursive spawn is just the span of a single one of them, or $M_\infty(n/2)$. The span for the doubly nested **parallel for** loops in lines 5–7 is $\Theta(\lg n)$ because each loop control adds $\Theta(\lg n)$ to the constant span of line 7. Similarly, the doubly nested **parallel for** loops in lines 18–20 add another $\Theta(\lg n)$. Matrix partitioning by index calculation has $\Theta(1)$ span, which is

dominated by the $\Theta(\lg n)$ span of the nested loops. We obtain the recurrence

$$M_\infty(n) = M_\infty(n/2) + \Theta(\lg n) . \quad (26.6)$$

Since this recurrence falls under case 2 of the master theorem with $k = 1$, the solution is $M_\infty(n) = \Theta(\lg^2 n)$.

The parallelism of P-MATRIX-MULTIPLY-RECURSIVE is $M_1(n)/M_\infty(n) = \Theta(n^3/\lg^2 n)$, which is huge. (Problem 26-2 asks you to simplify this parallel algorithm at the expense of just a little less parallelism.)

Parallelizing Strassen's method

To parallelize Strassen's algorithm, we can follow the same general outline as on pages 86–87, but use spawning. You may find it helpful to compare each step below with the corresponding step there. We'll analyze costs as we go along to develop recurrences $T_1(n)$ and $T_\infty(n)$ for the overall work and span, respectively.

1. If $n = 1$, the matrices each contain a single element. Perform a single scalar multiplication and a single scalar addition, and return. Otherwise, partition the input matrices A and B and output matrix C into $n/2 \times n/2$ submatrices, as in equation (4.2) on page 82. This step takes $\Theta(1)$ work and $\Theta(1)$ span by index calculation.
2. Create $n/2 \times n/2$ matrices $S_1, S_2, \dots, S_{10}$, each of which is the sum or difference of two submatrices from step 1. Create and zero the entries of seven $n/2 \times n/2$ matrices $P_1, P_2, \dots, P_7$ to hold seven $n/2 \times n/2$ matrix products. All 17 matrices can be created, and the P_i initialized, with doubly nested **parallel for** loops using $\Theta(n^2)$ work and $\Theta(\lg n)$ span.
3. Using the submatrices from step 1 and the matrices $S_1, S_2, \dots, S_{10}$ created in step 2, recursively spawn computations of each of

the seven $n/2 \times n/2$ matrix products $P_1, P_2, \dots, P_7$, taking $7T_1(n/2)$ work and $T_\infty(n/2)$ span.

4. Update the four submatrices $C_{11}, C_{12}, C_{21}, C_{22}$ of the result matrix C by adding or subtracting various P_i matrices. Using doubly nested **parallel for** loops, computing all four submatrices takes $\Theta(n^2)$ work and $\Theta(\lg n)$ span.

Let's analyze this algorithm. Since the serial projection is the same as the original serial algorithm, the work is just the running time of the serial projection, namely, $\Theta(n^{\lg 7})$. As we did with P-MATRIX-MULTIPLY-RECURSIVE, we can devise a recurrence for the span. In this case, seven recursive calls execute in parallel, but since they all operate on matrices of the same size, we obtain the same recurrence (26.6) as we did for P-MATRIX-MULTIPLY-RECURSIVE, with solution $\Theta(\lg^2 n)$. Thus the parallel version of Strassen's method has parallelism $\Theta(n^{\lg 7} / \lg^2 n)$, which is large. Although the parallelism is slightly less than that of P-MATRIX-MULTIPLY-RECURSIVE, that's just because the work is also less.

Exercises

26.2-1

Draw the trace for computing P-MATRIX-MULTIPLY on 2×2 matrices, labeling how the vertices in your diagram correspond to strands in the execution of the algorithm. Assuming that each strand executes in unit time, analyze the work, span, and parallelism of this computation.

26.2-2

Repeat Exercise 26.2-1 for P-MATRIX-MULTIPLY-RECURSIVE.

26.2-3

Give pseudocode for a parallel algorithm that multiplies two $n \times n$ matrices with work $\Theta(n^3)$ but span only $\Theta(\lg n)$. Analyze your

algorithm.

26.2-4

Give pseudocode for an efficient parallel algorithm that multiplies a $p \times q$ matrix by a $q \times r$ matrix. Your algorithm should be highly parallel even if any of p , q , and r equal 1. Analyze your algorithm.

26.2-5

Give pseudocode for an efficient parallel version of the Floyd-Warshall algorithm (see Section 23.2), which computes shortest paths between all pairs of vertices in an edge-weighted graph. Analyze your algorithm.

26.3 Parallel merge sort

We first saw serial merge sort in Section 2.3.1, and in Section 2.3.2 we analyzed its running time and showed it to be $\Theta(n \lg n)$. Because merge sort already uses the divide-and-conquer method, it seems like a terrific candidate for implementing using fork-join parallelism.

The procedure P-MERGE-SORT modifies merge sort to spawn the first recursive call. Like its serial counterpart MERGE-SORT on page 39, the P-MERGE-SORT procedure sorts the subarray $A[p : r]$. After the **sync** statement in line 8 ensures that the two recursive spawns in lines 5 and 7 have finished, P-MERGE-SORT calls the P-MERGE procedure, a parallel merging algorithm, which is on page 779, but you don't need to bother looking at it right now.

```
P-MERGE-SORT ( $A, p, r$ )
1 if  $p \geq r$  // zero or one element?
2   return
3  $q = \lfloor (p + r)/2 \rfloor$  // midpoint of  $A[p : r]$ 
4 // Recursively sort  $A[p : q]$  in parallel.
5 spawn P-MERGE-SORT ( $A, p, q$ )
6 // Recursively sort  $A[q + 1 : r]$  in parallel.
7 spawn P-MERGE-SORT ( $A, q + 1, r$ )
8 sync // wait for spawns
```